the following command:

```
osfy.test        #from /snap/bin
```

For listing of installed snaps, use the command given below:

```
snap list
```

To check the information about a particular installed snap, use the following command:

```
snap info osfy
```

The following command updates all installed snaps:

```
\sudo snap refresh
```

The following command updates a particular installed snap:

```
sudo snap refresh osfy
```

To roll back a snap to a previous version, use the command given below:

```
sudo snap revert  osfy
```

The next command uninstalls a specific snap:

```
sudo snap remove osfy
```

The following command removes symlinks of the apps in */snap/bin* and stops the associated services:

```
sudo snap disable osfy
```

To start the snap services and make symlinks for the apps available in */snap/bin*, use the following command:

```
sudo snap enable osfy
```

To list the environment variables associated with a snap, use the command given below:

```
sudo snap run osfy.env
```

Classic snap

Since the core is read-only and not meant for development purposes, you can install 'classic snap' to enter in classic mode on devices deployed with solely Ubuntu Core. Normal package management with apt-get is allowed in classic mode. But avoid this classic snap if secure deployment is the prime concern.

```
sudo snap install --devmode classic
sudo classic
```

Now, in the prompt enabled by classic mode, you can try any developer commands such as those shown below:

```
sudo apt-get update
sudo apt-get install vim gcc snapcraft
```

Interfaces

Interfaces allow snaps to exchange services with each other. A snap provides the service in the form of *slot* and another snap can consume it in the form of *plug*. Some important interfaces are *network*, *network-bind*, *camera*, *bluetooth-control*, *firewall-control*, *log-observe*, etc. To list out all possible interfaces and consumer snaps plugged into each of them, use the following command:

```
snap interfaces
```

To list interfaces plugged by a particular snap, the following command should be used:

```
snap interfaces osfy
```

In the absence of a *plugs* entry in the *apps* section of the manifest file, we need to connect the interfaces manually. For example, if a *foo* snap is providing a *bar* interface slot and *osfy* snap wants to plug into it, run the following command:

```
snap connect osfy:bar foo:bar
```

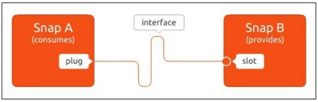

Figure 2: Snap interfaces

Building custom snaps

You can package any Linux application as a snap. For this, create an empty directory and run the following command to create the basic skeleton of metadata in the name of *snapcraft.yaml*:

```
snapcraft init
```

Now edit the *parts*, *apps* sections of *snapcraft.yaml* as per the tutorial in *https://snapcraft.io/docs/build-snaps/your-first-snap*. And, finally, run the *snapcraft* command to build the snap by retrieving all required sources, building each part, and staging and priming the outcome of all parts in stage, prime directories. Finally, the contents of the prime sub-directory will be bundled